
Modül 10 — Elasticsearch ile Arama

Ödevler

Spring Boot Edu



2026-09-23

İçindekiler

Nasıl Çalışılır	2
Ödev 1 — Otomatik Tamamlama (Kolay)	2
Ödev 2 — İşe Yarar Kalan Facet'ler (Orta)	3
Ödev 3 — Kesintisiz Yeniden İndeksleme (Zor)	3
Ek Meydan Okuma (İsteğe Bağlı)	4

Nasıl Çalışılır

1. Başlangıç kodu `modules/10-elasticsearch/exercise/` altındadır. TODO yorumlarını bulun.
2. Testler gerçek bir Elasticsearch'e (Testcontainers) karşı çalışır. **Docker açık olmalı** ve en az 2 GB belleğe sahip olmalıdır.
3. Testleri çalıştırın:

```
./mvnw -Pexercises -pl modules/10-elasticsearch/exercise -am test
```

4. Tüm testler yeşil olduğunda ödev tamamdır.
5. Takılırsanız önce ipuçlarını, sonra `modules/10-elasticsearch/solution/` altındaki çözümü okuyun.

★ İpucu

Her test sınıfı `@BeforeEach` içinde kendi indeksini siler ve yeniden oluşturur. Böylece ödevler birbirinden bağımsızdır.

Ödev 1 — Otomatik Tamamlama (Kolay)

Hedef: Kullanıcı daha yazarken kitap başlıkları önermek: “spr” yazınca “Spring in Action” bulunmalı.

Yapılacaklar (exercisel paketi):

- TODO 1a — `TitleDocument`: `title` alanını `search-as-you-type` alanı olarak eşleyin. Elasticsearch bu durumda kelime ikilileri ve üçlülere için `title._2gram` ve `title._3gram` alt alanlarını oluşturur.
- TODO 1b — `TitleSuggestions.suggest`: `title`, `title._2gram` ve `title._3gram` üzerinde `bool_prefix` tipinde bir `multi_match` sorgusu. Son kelime yarım olabilir.
- TODO 1c — En fazla limit sonuç. Yalnızca başlıkları döndürün.

İpuçları:

- `@Field(type = FieldType.Search_As_You_Type)`.
- `q.multiMatch(m -> m.query(typed).type(TextQueryType.BoolPrefix).fields(...))`.
- Ders bölüm 3.3, bir `NativeQuery`'nin nasıl çalıştırılacağını ve sonuçlarının nasıl okunacağını gösterir. `withMaxResults(limit)` sonuç sayısını sınırlar.

Kabul kriterleri: `ExerciselTest` içindeki 5 testin hepsi geçer.

Tahmini süre: 25 dakika

Ödev 2 — İşe Yarar Kalan Facet'ler (Orta)

Hedef: Bir mağaza arama sayfası. Kullanıcı “books” kategorisini seçtikten sonra sonuçlar yalnızca kitapları gösterir, ancak kategori listesi hâlâ kaç “kitchen” ve “stationery” ürününün eşleştiğini gösterir. Aksi hâlde kullanıcı başka bir kategoriye asla geçemezdi.

Yapılacaklar (exercise2.FacetedSearch.search):

- TODO 2a — name alanı metinle eşleşen ürünler. maxPrice verilirse yalnızca bu fiyata kadar olan ürünler.
- TODO 2b — category alanı üzerinde categories adlı bir terms aggregation'ı.
- TODO 2c — Seçilen kategori **sonuçları** filtrelemeli, ama kategori sayılarını **filtrelememeli**dir.
- TODO 2d — Sonuçları ve kategori başına sayıları döndürün.

İpuçları:

- Elasticsearch aggregation'ları **sorgunun** sonucu üzerinde hesaplar. Bir **post filter** sonradan uygulanır ve yalnızca sonuçları eler.
- NativeQueryBuilder.withFilter(Query) bir post_filter gönderir. Fiyat koşulu sorguya (bool + filter) aittir, çünkü sayıları da değiştirmelidir.
- Ders bölüm 3.4, bir terms aggregation'ının bucket'larının nasıl okunacağını gösterir.

Kabul kriterleri: Exercise2Test içindeki 4 testin hepsi geçer.

Tahmini süre: 40 dakika

Ödev 3 — Kesintisiz Yeniden İndeksleme (Zor)

Hedef: Var olan bir indekste bir alanın eşlemesi değiştirilemez. Değiştirmek için yeni bir indeks kurup tüm veriyi ona kopyalamak gerekir, üstelik aramanın hiçbir şey bulamadığı bir an olmadan.

Püf noktası bir **alias**'tır: Uygulama her zaman catalog üzerinde arar. Bu, gerçek indeks catalog-<ek>'i gösteren ikinci bir addır. Yeniden indeksleme eskisinin yanına yeni bir indeks kurar ve alias'ı tek bir atomik adımda taşır.

Yapılacaklar (exercise3.Reindexer.reindex):

- TODO 3a — CatalogBook'un ayarları ve eşlemesiyle yeni bir catalog-<benzersiz ek> indeksi oluşturun.
- TODO 3b — BookSource'taki her kitabı ona yazın ve refresh edin.
- TODO 3c — catalog alias'ının şu an gösterdiği indeksleri bulun. İlk çalıştırmada hiç yoktur.
- TODO 3d — **Tek** bir alias isteğiyle alias'ı yeni indekse ekleyin ve eskilerinden kaldırın.
- TODO 3e — Eski indeksleri silin ve yenisinin adını döndürün.

İpuçları:

- `IndexOperations ops = operations.indexOps(IndexCoordinates.of(name))`, ardından `ops.create(ops.createSettings(CatalogBook.class), ops.createMapping(CatalogBook.class))` tam olarak o indekse yazar.
- `operations.save(books, IndexCoordinates.of(name))` tam olarak o indekse yazar.
- `operations.indexOps(IndexCoordinates.of(ALIAS)).exists()` bir alias için de true döner. `getAliases(ALIAS)`, indeks adından alias verisine bir map döndürür.
- `new AliasActions(new AliasAction.Add(...), new AliasAction.Remove(...))` ve `AliasActionParameters.builder().withIndices(...).withAliases(ALIAS).build();` `ops.alias(actions)` ile gönderin.
- İndeks adları küçük harf olmalıdır ve aynı milisaniyedeki iki çalıştırma çakışmamalıdır.

Kabul kriterleri: Exercise3Test içindeki 3 testin hepsi geçer.

Tahmini süre: 60 dakika

Ek Meydan Okuma (İsteğe Bağlı)

Derse, Reindexer'ımızı PostgreSQL'deki kitaplara karşı çalıştıran bir POST `/api/admin/reindex` endpoint'i ekleyin. Yeniden indeksleme sürerken kaydedilen kitaplara ne olur? Bunların kaybolmamasını sağlamanın iki yolunu anlatın.